Underworld 2

Louis Moresi [1]

[1] Australian National University

DOI https://doi.org/10.6084/m9.figshare.33193233

Keywords Underworld Code

(aka the Underworld renovation project)

Underworld is our parallel, particle-in-cell, finite element geodynamics code [1]. For the past year or so, the Underworld team has been working on a refurbished user interface. We've known for a long time that it needed to be done but we finally bit the bullet.

We settled on turning underworld into a python code for two main reasons: 1) because many of our dependencies already have python bindings and 2) as we move more towards integrating with observations, it is necessary to interface with packages to pre-process data, many of which are python based or wrapped. I just thought of another one: 3) because we could. The tools for wrapping codes into python make this job much less scary than before.

1. Background

The Underworld / StGermain C code base has been around for about 10 years now, has been extensively debugged and is known to work well on hundreds to thousands of processors. We have engineered some pretty robust solver methods and lots of flexibility in dealing with complex rheology. I was pretty nervous about disrupting this part of the code no matter how much we needed to improve the user-interface.

Underworld is based on the StGermain framework [2] which is modular and object oriented; all important entities within the code (meshes, for example) being objects that allow proper inheritance and multiple, independent copies. Unfortunately, the objects have always needed to be described in largely-static xml files which have caused many levels of anxiety and trauma for the users. Developers have also had trouble with the complexity of the object model implemented in C and relatively simple code hacks by smart users (i.e. geophysicists !) were virtually impossible.

We have been through a number of different iterations on how to improve the user experience for newcomers and experts alike. We partnered with CIG to try to build a user-friendly version of underworld but, in the end, our various experiments with different approaches to running models have not significantly changed how people use the code.

PUBLISHED Aug 11, 2015 ARTICLE UWTN 2015-001 VERSION 1.0.0 LICENCE CC-BY-4.0 ARCHIVED 2026-08-10T04:19:43Z SOURCE https://www.underworldcode.org/underworld-2/

I have to admit that I was becoming concerned that we would not be in a position to support our users and maintain the code in future without taking steps to build a workable open-source community of user / developers (as happened with Citcom). John Mansour convinced me that a python interface would be a workable environment for geophysicist users to develop new functionality by building from low-level, parallel-efficient, underworld bricks. I didn't really believe him but the power of `swig` to build a prototype interface in a few days proved he was right (and I was wrong).

The first iteration was simply to wrap all the functions of the underworld layers with `swig` and use python to glue them together in various ways. Much better than xml, but we were still populating a dictionary and passing around C objects which we could not open directly in python.

We began to bundle up the component declarations within python functions and built modules to create common patterns of usage in the code. Gradually we added ways to interact with the live objects while the code was running which allowed us to do simple things like monitoring values and changing certain parameters at runtime. We developed a sketch of an interface and began to use it for teaching and revisiting benchmark problems to see how well the approach worked in practice.

Meanwhile John began gutting the underlying code and implementing a tight, low level integration of underworld objects as python objects and a numpy interface to underworld data structures. The alpha version of the code is in the internal testing phase and we will release it on github in the next few days.

2. Some highlights

Issuing the command

```
import underworld
```

in the python or ipython interpreter will fire up the underworld subsystem. Underworld is still a geodynamics finite element code, so you find classes and methods which provide relevant FE machinery, interfaces to the equation systems we expect to encounter, and particle manipulation.

For example, here is how we define a Cartesian mesh of Q1P0 elements

```
# New Q1P0 element mesh

Q1P0Mesh = uw.mesh.FeMesh_Cartesian( elementType=("linear", "constant"),
                                     elementRes=(meshX,meshY),
                                     minCoord=(0.,0.), maxCoord=(1.,1.)  )


velocityMesh =  Q1P0Mesh             # behaves as outer mesh by default
pressureMesh =  Q1P0Mesh.subMesh
```

and define variables on the mesh which slot into the Stokes (velocity / pressure) solver and the advection-diffusion solver that is used to update the temperature or other diffusive quantities.

```
# define some mesh-based variables
velocityField    = uw.fevariable.FeVariable( feMesh=velocityMesh,   nodeDofCount=dim )
pressureField    = uw.fevariable.FeVariable( feMesh=pressureMesh,   nodeDofCount=1 )
temperatureField = uw.fevariable.FeVariable( feMesh=velocityMesh,   nodeDofCount=1 )
```

```
# and provide them to Stokes and advection / diffusion solvers
stokesEqn = uw.systems.Stokes(velocityField=velocityField,
                              pressureField=pressureField,
                              conditions=[freeslipBC,],
                              viscosityFn=fn.exception.SafeMaths(viscosityFn),
                              bodyForceFn=forceFn )

advDiff   = uw.systems.AdvectionDiffusion( temperatureField=temperatureField,
                                           velocityField=velocityField,
                                           diffusivity=1.,
                                           conditions=[tempBC,] )
```

The numpy interface is used to set and retrieve values of the FE variables (mesh variables which know about interpolation and integration strategies)

```
for index, coord in enumerate(velocityMesh.data):
    pertCoeff = math.cos( math.pi * coord[0] ) * math.sin( math.pi * coord[1] )
    tempData[index] = min(1,max(0,toptemp + scaleFactor*(1. - coord[1]) + perturbation_scale *
pertCoeff ));

# boundary condition values on horizontal

for index in velocityMesh.specialSets["MaxJ_VertexSet"]:
    temperatureField.data[index] = 0
```

A function interface is provided to evaluate expressions containing FEvariables. Function evaluation is very lazy indeed and is computed at whatever spatial points are passed into the function. Functions are parallel safe whereas the numpy interface is not.

```
# define viscosity for StokesEqn
viscosityFn = eta0 *  fn.math.exp( activationEnergy / (temperatureField+1.) )
```

Timestepping is an explicit loop on the underworld machinery

```
    while step<500:
        # Get solution for initial configuration
        stokesEqn.solve()
        # Retrieve the maximum possible timestep for the AD system.
        dt = advDiff.get_max_dt()
        if step == 0:
            dt = 0.
        # Advect using this timestep siz
        advDiff.integrate(dt)
        advector.integrate(dt)
```

I could go on, but this is not a manual, just a guidebook to the flavour of the new code.

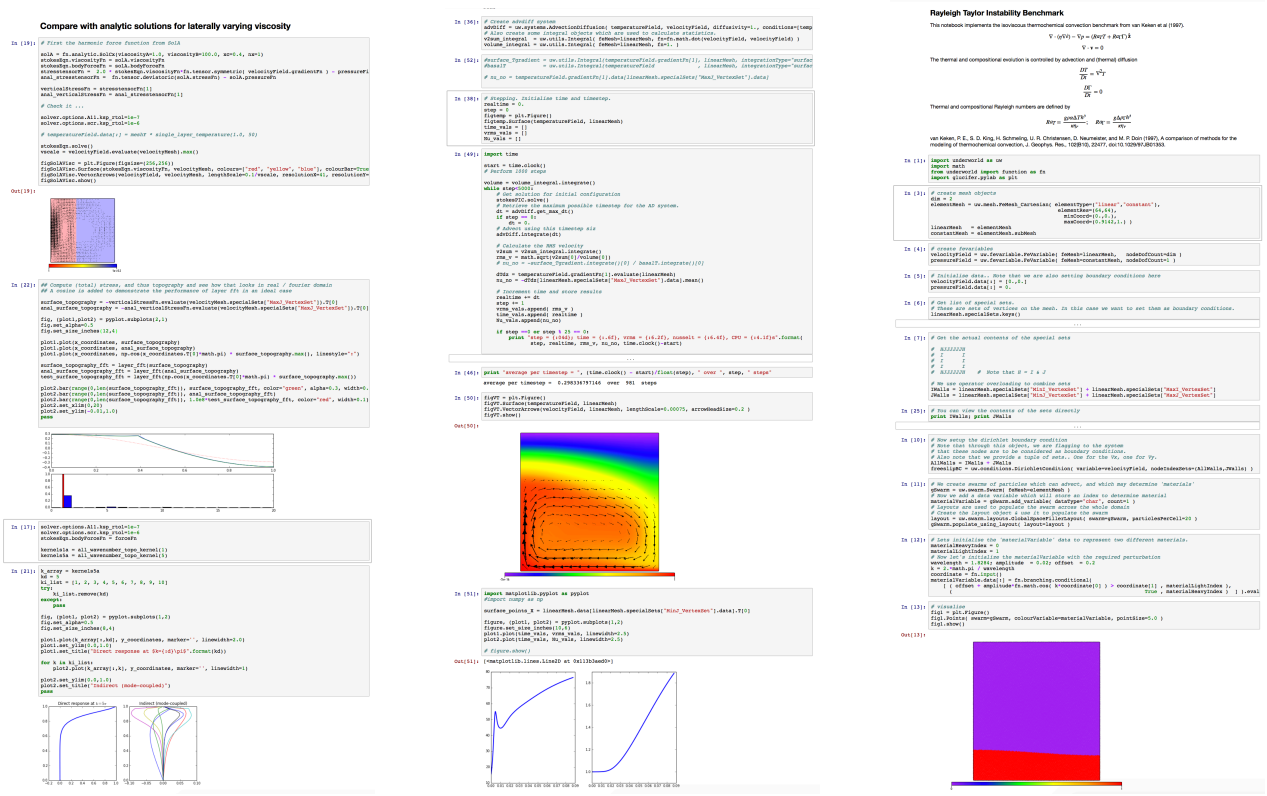
3. Teaching tools (underworld for the masses)

A side effect of wrapping Underworld is that we can use it within the 'literate' programming enviroment provided by the ipyton / jupyter notebook system. In the last year, this project has grown dramatically in confidence and is now a rock solid programming environment that also provides the ability to intermingle markdown (github style, naturally) with rendered mathematics and live code (in python or julia primarily).

The notebooks live within the familiar envirnment of a web browser whether running locally or on a remote machine or on a virtual machine running wherever the user decides. This is a liberating environment for teaching a class or providing self-directed learning examples with the code.

We have been focussing strongly on underworld in the notebook to try to increase the number of people who are able to run simple examples. We have to remember that the i in ipython stands for 'interactive' and the notebooks were a development of the interactive python extensions. This is not such a great pathway for setting up large scale models to run in a batch environment but ... 1) crawl first, then walk, 2) things are changing fast and it may be the batch environment that disappears ... so we'll see.

Here are some examples of the code running in notebook form:



(a) (b) (c)

Figure 1: Examples of the notebooks in action. Note the mixture of mathematical explanation and documentation with runnable code and outputs including visualisation and analysis. The code can be interupted at any stage and the progress can be analysed either interactively or automatically.

One of these reproduces a big chunk of my DPhil thesis work in a page or two.

4. Parallelism and HPC

The one advantage of the xml / static instantiation approach from the older version of Underworld is that the parallelism can almost entirely be hidden from the users because they can only configure existing (parallel-aware) plugins at run time. (Of course, to add new functionality requires cracking open the parallel, C code and rewiring things).

However, once the interface to the code becomes a scripting language, the user has to manage the data directly - for example to set up initial conditions, to analyse results while the code is live, or to couple with other software packages. If the user is not aware that they are operating in an MPI parallel environment then he/she will not realise that any individual script only see part of the data.

Underworld provides an enormous and reliable collection of data objects and operations which are safe in a multi-processor environment and handle all the parallel operations correctly / efficiently.

Our goal is, of course, to minimise the opportunities for parallel bugs to manifest in the python layer. To this end we provide a high-level and powerful function interface which we encourage users to use in place of directly handling the underlying data. The function interface is safe in parallel with the trade off that it sometimes can be more convoluted to write trivial operations. (Tough luck !)

This particular aspect of Underworld2 is not fully cooked yet. It's something we can probably only develop when we see what trips people up. We'll be encouraging users to give us feedback on the problems they encounter over the next few months.

5. References

1. Moresi, L. N., S. Quenette, V. Lemiale, C. Mériaux, B. Appelbe, and H. B. Muhlhaus (2007), Computational approaches to studying non-linear dynamics of the crust and mantle, Physics of the Earth and …, 163(1-4), 69–82, doi:10.1016/j.pepi.2007.06.009.
2. Quenette, S., Moresi, L.N., Sunter, P.D., Appelbe, W.F., 2007. Explaining StGermain: An aspect oriented environment for building extensible computational mechanics modeling software, in:. Presented at the HIPS 2007 Workshop, Parallel and Distributed Processing Symposium, 2007. Proceedings. 19th IEEE International.